CHAPTER 17

Proximity Service

Find nearby businesses (restaurants, hotels, gas stations) for a given location and radius - Yelp / Google Places.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | API / data model | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a **proximity service** that discovers nearby places. Users provide their location (lat/long) and a search radius (0.5 km, 1 km, 2 km, 5 km, 20 km) and get back a list of businesses. Business owners can add, update and delete listings (changes may appear next day). Users can view a business's details.

Why interviewers ask it: Core geospatial indexing question: geohash vs quadtree vs S2, read-heavy architecture and read/write separation.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Max radius? Search radius options?	20 km max; 0.5, 1, 2, 5, 20 km selectable.
Business updates reflected immediately?	No; next-day is fine.
Scale?	100 M DAU, 200 M businesses.
Moving user?	Out of scope (static location per search).

3. Requirements

Functional

- Return all businesses within a radius of a given location.
- Add/update/delete/view business info.
- View business details.

Non-functional

- Low latency for search.
- Data privacy compliance (location data).
- High availability and scalability for peak hours in dense areas.

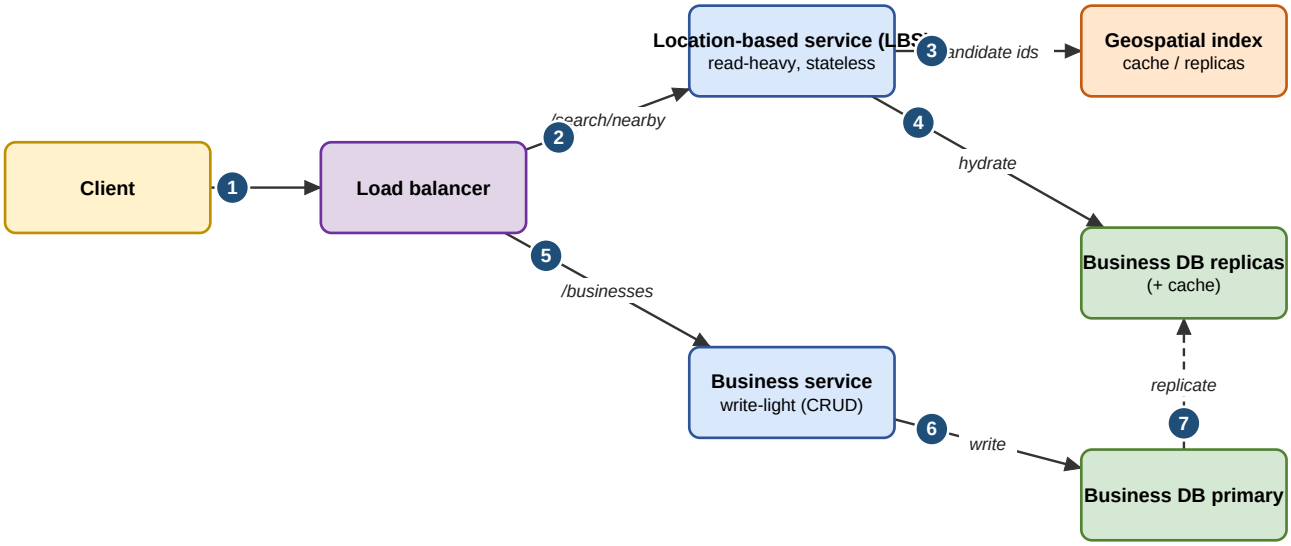
4. Back-of-the-envelope estimation

- 100 M DAU x 5 searches/day = 500 M searches/day = about **5,000 search QPS** (peak 2x).
- 200 M businesses x about 1 KB = 200 GB of business data; the geospatial index (id + lat/long) is small: about 200 M x 24 B = about 5 GB -> fits in memory on a single server (replicate for availability).
- Business updates are low-volume; reads dominate by 1000:1.

5. API / data model

```
GET /v1/search/nearby?latitude=..&longitude=..&radius=500        -> { total, businesses: [ {id, name, ...} ] }
GET /v1/businesses/{id}      POST /v1/businesses      PUT /v1/businesses/{id}      DELETE /v1/businesses/{id}
Tables: business(id, address, city, state, lat, long, ...)   geo_index(geohash, business_id)
```

6. High-level design



Separate the read-heavy search path (LBS) from the write-light business CRUD so each scales independently.

Step	What happens
1	The client sends `GET /v1/search/nearby?latitude=..&longitude=..&radius=500` to the load balancer.
2	Search requests go to the **location-based service (LBS)**: read-heavy, stateless, scaled out.
3	The LBS picks the geohash precision for the radius, computes the user's cell **plus its 8 neighbours**, and fetches **candidate business IDs** from the geospatial index (Redis or a table; small, so replicated, not sharded).
4	It **hydrates** the candidates from the business DB read replicas / cache, computes exact distances, filters beyond the radius, sorts and paginates.
5	Separately, owners call `POST/PUT/DELETE /businesses`, which go to the **business service** (write-light).
6	Writes go to the **primary** business DB.
7	The primary replicates to read replicas. The geospatial index is updated nightly (or via change stream), since next-day freshness is acceptable.

7. Deep dive

Why not a plain SQL range query?

- `WHERE lat BETWEEN .. AND long BETWEEN ..` needs two one-dimensional index scans and intersects huge result sets. Indexes are 1-D; we need a 2-D structure.

Option 1: Even grid

- Divide the world into fixed cells. Simple, but data is uneven (Manhattan vs Pacific) -> some cells huge, most empty.

Option 2: Geohash (recommended, simple)

- Encodes (lat, long) into a base-32 string by repeatedly bisecting the world; **longer prefix = smaller cell** (len 4: about 39 x 20 km; 5: about 5 x 5 km; 6: about 1.2 x 0.6 km). Pick the longest precision whose **cell is still at least as large as the radius**: 0.5 / 1 / 2 / 5 / 20 km -> precision 6 / 5 / 5 / 4 / 4.
- Nearby places share a prefix, so index `geohash` with a B-tree / SQL `LIKE 'prefix%'`, or use `geohash -> [business ids]` in Redis.
- **Boundary issues:** two close points can have different prefixes (across a cell edge) and distinct cells with the same prefix can be far apart. Fix: **query the 8 neighbouring cells** too.
- Not enough results? Drop a character (increase the radius) and retry. Finally filter by exact distance and sort.

Option 3: Quadtree

- Recursively split a node into 4 quadrants until each has <= 100 businesses (adaptive: dense areas deeper). In-memory tree (about 1.7 GB for 200 M businesses at 100 per leaf) built at server start.
- **Pros:** adaptive, k-nearest friendly. **Cons:** build time at startup, updates require tree changes, higher complexity than geohash.

Option 4: Google S2 / Hilbert curves

- Maps the sphere to 1-D via a space-filling curve; supports **region covering**, great for geofencing; more complex. Geohash is easier to explain in interviews; S2 is used by Google/Tinder.

Data and scaling

- Business table sharded by `business_id`; read replicas. Geospatial index table: `(geohash, business_id)` rows - **do not shard** (small), replicate instead; a business can be updated in the index with simple INSERT/DELETE.
- Cache hot data (Redis). Regional deployment with users routed to nearest region (and some countries require data locality).
- Filtering by category/open-now: apply after fetching candidates or use secondary indexes.

Nightly index update

- Because next-day freshness is acceptable, rebuild the geospatial index/quadtree nightly (or incrementally via change stream) and swap atomically.

8. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Index	Geohash / Quadtree / S2	Geohash for simplicity & DB friendliness; quadtree for variable density; S2 for advanced geo features.
Index storage	In-memory tree on each LBS / shared Redis / DB table	Small data -> replicate per LBS server; shared store simplifies updates.
Radius vs precision	Fixed precision per radius / adaptive widening	Widen the cell until enough results.

9. Scaling, reliability, failure modes

- LBS is stateless: scale out behind LB; add replicas of index and DB.
- Dense-city hotspots: finer geohash precision / quadtree adaptivity.
- Cross-region: multi-region deployment, GeoDNS.

10. Likely follow-up questions

- How do you return results sorted by distance and by rating?
- How do you handle a moving user (Nearby Friends - chapter 18)?
- What if a radius search at a geohash border returns too few results?
- How would you shard when the geo index grows 100x?

11. Common mistakes

- Using a SQL 2-D range query on separate lat/long B-tree indexes.
- Forgetting neighbour cells in geohash (boundary issue).
- Sharding the tiny geospatial index needlessly.

12. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"I'll assume the user gives a latitude, longitude and a radius up to 20 km, with 100 M DAU and 200 M businesses. Business updates may appear next day. That gives about 5,000 search QPS and only a few writes, so this is a very read-heavy system, and the geospatial index is small enough to hold in memory."

Phase	Time	What to say
2. High-level design	10 min	"I split into two services behind a load balancer: a read-heavy location-based service for nearby search and a write-light business service for CRUD. The business DB has a primary and read replicas, with a cache in front, and the geospatial index is replicated."
3. Deep dive	25 min	"A plain SQL range query on latitude and longitude is inefficient because indexes are one-dimensional. I'd use geohash: nearby points share a prefix, so I pick a precision where the cell is at least as large as the radius, query the user's cell plus the 8 neighbours to fix boundary problems, then filter by exact distance. The alternative is a quadtree, which adapts to density and gives k-nearest queries, but is harder to update. S2 is the heavy-duty option."
4. Wrap up	4 min	"I'd scale with replicas and caching, deploy per region, and handle dense city hotspots with finer geohash precision or a quadtree. Moving users are covered in Nearby Friends."

13. 60-second recap / practice checklist

- Two services: read-heavy LBS and write-light business CRUD.
- Geohash (+8 neighbours) or quadtree for candidates, then exact-distance filter + rank.
- Replicate index; cache hot; nightly updates acceptable.

14. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Geohash (Wikipedia)
• S2 Geometry library
• Quadtree (Wikipedia)
• GeeksforGeeks: Geohashing and quadtrees